

# Machine Learning para Finanzas

## Clase 1

César Núñez Cuevas

cesarnunezcuevas@gmail.com



UNIVERSIDAD  
**Finis Terrae**

# Me Presento

- Mi nombre es César Núñez
- Ingeniero Comercial & Master en Finanzas de la Universidad de Chile.
- Master en Business Analytics & Big Data en MIP Politecnico di Milano.
- 6 años de experiencia como Analista de Riesgo Financiero en Bci, McKinsey y Tanner Servicios Financieros.
- Experiencia como profesor en la Universidad de Chile y en la Universidad Santo Tomás.



# Sobre el curso

- El programa está diseñado para cerrar la brecha entre la ciencia de datos y la gestión financiera. Buscamos que el profesional no solo ejecute un algoritmo, sino que entienda el impacto de cada predicción en el balance y la solvencia de la institución.

| Unidad   | Tópicos                                          |
|----------|--------------------------------------------------|
| Unidad 1 | Fundamentos y Procesamiento de Datos Financieros |
| Unidad 2 | Modelado Supervisado para Riesgo Crediticio      |
| Unidad 3 | Aprendizaje Profundo y Análisis Multidimensional |
| Unidad 4 | Inteligencia Artificial Avanzada y Despliegue    |

# Evaluaciones

- Solemne 1: Prueba de desarrollo que aborda los contenidos hasta la sesión 5. Valdrá un 30 %
- Avance 1 del Proyecto: Entrega de propuesta inicial de un análisis de datos (EDA, definición de proyecto de inversión)
- Avance 2 del Proyecto: Entrega de avance que contenga elección de modelo, fine tuning e interpretación
- Presentación Final: Presentación de resultados finales. Valdrá 40 %

$$NotaFinal = \begin{cases} Nota_P & Nota_P > 5,5 \\ 0,7 \cdot Nota_P + 0,3 \cdot Nota_E & Nota_P \leq 5,5 \end{cases}$$

- Nota mínima de aprobación: 3.95 en escala de 1.0 a 7.0
- Asistencia mínima requerida: Ejemplo: 75
- Eximición: nota mayor o igual de 5.5. Con solemne 1, avances y proyecto con nota mayor o igual a 4.0

- El profesor no debe pedir justificantes por inasistencia a la solemne; la falta se traslada
- No se borran notas, de ningun tipo.
- Plagios parciales y/o totales se castigan con la nota minima en la evaluacion afectada.

- Las clases serán expositivas y de discusión, el material estará habilitado previo a la realización de la clase.
- Si me necesitan contactar, me envían un correo si no contesto en 24hrs me insisten.

# Cronograma de Laboratorios de ML con Python

El curso sigue el siguiente calendario de sesiones y temas:

- Clase 1: El Entorno de Trabajo y Variables Financieras
- Clase 2: Manipulación de Datos Financieros con Pandas
- Clase 3: Visualización y Análisis Exploratorio (EDA)
- Clase 4: Regresión Logística y Probabilidad de Default (PD)
- Clase 5: Árboles de Decisión y Random Forests
- Clase 6: Métodos de Ensamble y Boosting (XGBoost/LightGBM)
- Clase 7: Aprendizaje No Supervisado (Clustering y PCA)
- Clase 8: Redes Neuronales Artificiales (MLP)
- Clase 9: Deep Learning para Secuencias (LSTM)
- Clase 10: Reinforcement Learning (RL) - El Agente de Trading
- Clase 11: LLMs y Análisis de Sentimiento Financiero

**Material del curso:** <https://github.com/cnunezc/MFLUFT02026>.



El lenguaje de programación principal.

**Sirve para:**

- Automatizar cálculos financieros
- Procesar grandes volúmenes de datos
- Crear modelos cuantitativos y algoritmos de trading
- Integrar machine learning en finanzas

Es el entorno base donde corren todas las bibliotecas.





Entorno interactivo (notebook).

**Sirve para:**

- Ejecutar código, ver resultados y gráficos inmediatamente
- Documentar análisis con texto, ecuaciones y código
- Compartir fácilmente (Colab es gratuito y en la nube)
- Ideal para exploración de datos financieros y prototipado rápido



Biblioteca para computación numérica.

**Sirve para:**

- Manejar arrays y matrices de forma eficiente
- Realizar operaciones matemáticas rápidas (vectores, retornos, matrices de covarianza)
- Base para casi todas las bibliotecas científicas en Python



Biblioteca para manipulación y análisis de datos tabulares.

**Sirve para:**

- Manejar series temporales (precios, volúmenes)
- Limpiar datos financieros
- Calcular retornos, medias móviles, rolling windows
- Agrupar, filtrar y unir datasets (e.g. precios + fundamentales)



Biblioteca base para visualización en 2D.

**Sirve para:**

- Crear gráficos personalizados de precios, retornos, histogramas
- Visualizar series temporales financieras
- Hacer plots técnicos (velas, indicadores)
- Muy flexible, pero requiere más código



Biblioteca de visualización estadística (basada en Matplotlib).

**Sirve para:**

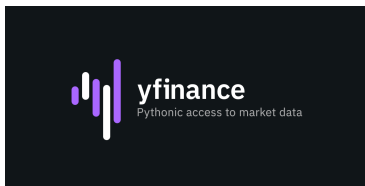
- Crear gráficos estadísticos atractivos con muy poco código
- Heatmaps de correlaciones (matriz de activos)
- Boxplots, violin plots, pairplots para analizar distribuciones
- Mejora la estética y facilita visualización exploratoria de datos financieros



Biblioteca de Machine Learning clásica.

**Sirve para:**

- Modelos de regresión para predecir precios o retornos
- Clasificación (e.g. predecir subidas/bajadas)
- Clustering de activos o detección de regímenes de mercado
- Validación cruzada, métricas, preprocesamiento
- Muy usada en estrategias cuantitativas y backtesting



Biblioteca para descargar datos financieros de Yahoo Finance.

**Sirve para:**

- Obtener precios históricos, dividendos, splits, datos fundamentales
- Descargar datos de acciones, ETFs, índices, criptomonedas, etc.
- Fácil acceso a datos gratuitos para backtesting y análisis
- Ejemplo: `yf.download('AAPL', start='2020-01-01')`

# Keras: Deep Learning de alto nivel en Python

## Keras

Es una API de alto nivel para construir y entrenar redes neuronales profundas (deep learning).

Desde 2017 forma parte oficial de TensorFlow como `tf.keras`, pero desde Keras 3 también soporta múltiples backends (TensorFlow, JAX, PyTorch).





# Keras: Deep Learning de alto nivel en Python

## Keras

### Para qué sirve en finanzas y análisis cuantitativo:

- Crear y experimentar rápidamente modelos de deep learning para predicción de precios, retornos o volatilidad.
- Modelos de series temporales: LSTM, GRU, CNN-LSTM para forecasting de acciones, índices, tipos de cambio, criptomonedas.
- Clasificación: predecir subidas/bajadas del mercado, señales de compra/venta, detección de anomalías.
- Regresión: predecir precios futuros, spreads, o variables macroeconómicas.
- Reinforcement Learning básico (combinado con otras libs) para optimización de portafolios dinámicos.
- Value-at-Risk (VaR), Expected Shortfall y modelado de riesgos con redes neuronales.
- Procesamiento de datos no estructurados: sentiment analysis de noticias/tweets para trading.

# Ejercicios Prácticos

Ejercicio 1: Funciones Crear una función que, dados dos números enteros  $m$  y  $n$ , calcule el máximo común divisor (GCD) entre ambos.

# Ejercicios Prácticos

## Ejercicio 2: NumPy

- 1 Crear un arreglo aleatorio de longitud 100. *Sugerencia:* `np.random.rand()`.
- 2 Ordenar el arreglo.
- 3 Calcular la media, la mediana y la varianza de la muestra.

# Ejercicios Prácticos

## Ejercicio 3: Pandas

- ➊ Importar el conjunto de datos *iris* como un DataFrame.
- ➋ Añadir los nombres de las columnas: *sepal length*, *sepal width*, *petal length*, *petal width*.
- ➌ Crear una nueva columna que contenga la relación (ratio) entre la longitud del sépalo y del pétalo.
- ➍ Añadir una nueva columna llamada *target* con valor 1 si el tipo es "setosaz" 0 en caso contrario.

## Importación de yfinance

- Instalar en Google Colab: `!pip install yfinance`
- Importar: `import yfinance as yf`
- Principal uso: Descargar datos históricos de acciones, ETFs, índices, criptos, etc. de Yahoo Finance de forma gratuita y sencilla. Link: <https://finance.yahoo.com/quote/AAPL/>

# Importación de yfinance

- Instalar en Google Colab: `!pip install yfinance`
- Importar: `import yfinance as yf`
- Principal uso: Descargar datos históricos de acciones, ETFs, índices, criptos, etc. de Yahoo Finance de forma gratuita y sencilla. Link: <https://finance.yahoo.com/quote/AAPL/>

```
# Instalación (solo la primera vez en Colab)
!pip install yfinance

# Importación
import yfinance as yf

# Ejemplo básico: descargar datos de AAPL
datos = yf.download('AAPL', start='2023-01-01', end='
    2023-12-31')

print(datos.head())
```

# Variables, Listas y Diccionarios en Python

- **Variables:** Almacenan un solo valor (número, texto, etc.)
- **Listas:** Colecciones ordenadas y mutables de elementos
- **Diccionarios:** Pares clave-valor (como un registro o JSON)

# Variables, Listas y Diccionarios en Python

- **Variables:** Almacenan un solo valor (número, texto, etc.)
- **Listas:** Colecciones ordenadas y mutables de elementos
- **Diccionarios:** Pares clave-valor (como un registro o JSON)

```
# Variable
activo = 'AAPL'
precio = 150.25

# Lista
precios = [140, 145, 150]

# Diccionario
info = {'ticker': 'AAPL', 'precios': precios}

print(info)

# Salida: {'ticker': 'AAPL', 'precios': [140, 145, 150]}
```



## Laboratorio: Descargar datos históricos de AAPL

- Usamos `yfinance` para obtener precios históricos
- `pandas` se importa automáticamente con `yfinance` (o explícitamente)
- Mostramos los primeros registros con `.head()`

## Laboratorio: Descargar datos históricos de AAPL

- Usamos yfinance para obtener precios históricos
- pandas se importa automáticamente con yfinance (o explícitamente)
- Mostramos los primeros registros con .head()

```
import yfinance as yf
import pandas as pd      # Buen hbito importarlo
                           expl citamente

# Descargar datos de AAPL
datos = yf.download('AAPL', start='2020-01-01', end='
2023-01-01')

# Mostrar los primeros 5 registros
print(datos.head())
```

# Laboratorio: Cálculo del Retorno Acumulado

- Retorno diario simple: `pct_change()`
- Retorno acumulado simple: producto acumulativo de  $(1 + \text{retorno diario})$
- `cumprod()` calcula el producto acumulativo

## Laboratorio: Cálculo del Retorno Acumulado

```
# Suponiendo que 'datos' ya est  cargado

# 1. Calcular retornos diarios simples
datos['Retorno_Diario'] = datos['Adj_Close'].
    pct_change()

# 2. Calcular retorno acumulado
datos['Retorno_Acumulado'] = (1 + datos['
    Retorno_Diario']).cumprod() - 1

# 3. Mostrar el retorno acumulado final
print("Retorno_acumulado_total:",
      datos['Retorno_Acumulado'].iloc[-1])

# Opcional: ver las  ltimas  filas
print(datos[['Adj_Close', 'Retorno_Diario', '
    Retorno_Acumulado']].tail())
```